



VIA University College

23/10/1944 - Projektrapport

Nikolaj Bræmer Christensen – 354322

Jonathan Cilius Østberg Winther Engell Gøtze – 354500

Victor Bruun Fassbender – 354361

PELC

Antal tegn:

XR Softwareingeniøruddannelsen

4. semester

27. maj 2026

Indhold

1	Resume	3
2	Indledning	4
3	Hovedafsnit	6
3.1	Metoder	6
3.1.1	Scrum-lignende udviklingsmetode	6
3.1.2	Teknologier og udviklingsværktøjer	7
3.1.3	Photogrammetry og map-udvikling	7
3.1.4	Dataindsamling og samarbejde med fagpersoner	8
3.1.5	Etiske overvejelser	8
3.2	Analyse	8
3.2.1	UserStories	8
3.2.2	Ikke funktionelle krav	12
3.2.3	Usecase Diagram	14
3.2.4	Actor Discription	16
3.2.5	Use case beskrivelser	16
3.2.6	Aktivitets diagrammer	16
3.2.7	System sequence diagrammer	16
3.2.8	State mashine diagrammer	16
3.2.9	Domaine model	16
3.3	Design	16
3.3.1	Klasse diagram	16
3.3.2	Sequence diagram	18
3.3.3	Conceptual ER-Diagram	19
3.3.4	EER-Diagram	20
3.3.5	Package Diagram	21
3.4	Implementation	22
3.4.1	Brug af OOP	22
3.4.2	Netværk	23
3.4.3	C++	23

3.4.4	Blueprints	24
3.5	Tests	24
3.5.1	Test Cases	24
3.6	Resultater	25
4	Diskussion	26
5	Konklusion og anbefalinger	27
	Litteratur	28

1 Resume

Dette projekt har til hensigt at besvare, hvordan man kan hylde fejringen af Viborg Katedralskoles 100-års jubilæum for bygningen ved hjælp af et spil. Med udgangspunkt i den historiske begivenhed den 23. oktober 1944, hvor Gestapo gennemførte en razzia på skolen og arresterede 14 modstandsfolk, udvikles en digital oplevelse, der formidler skolens besættelseshistorie til nuværende og kommende elever i aldersgruppen 14-19 år. Projektet anvender Scrum og Unified Process som styringsmetoder og udvikles i Unreal Engine med fokus på multiplayer-elementer, der muliggør samarbejde mellem elever. Gennem co-design med lærere og elever fra Katedralskolen integreres musik, historiske objekter og dilemma-baserede fortællinger. Resultatet er en spil-prototype, der demonstrerer, hvordan en historisk bygning kan fremvises i et historisk perspektiv, hvordan elever kan engagere sig gennem kreative tilføjelser, og hvordan en digital oplevelse kan designes med udgangspunkt i en konkret historisk begivenhed. Projektet bidrager med empirisk indsigt i anvendelsen af serious games til historieformidling i en skolekontekst.

2 Indledning

Den 23. oktober 1944 trængte Gestapo ind på Viborg Katedralskole og arresterede 14 modstandsfolk, mens flere andre måtte flygte (Viborg Katedralskole, 2026). Netop sådanne begivenheder kan gøre historien nærværende for elever, men erfaringer fra undervisningsverdenen tyder på, at potentialet sjældent forløses. En debattør med 40 års erfaring fra lærerseminariet beskrev i 2017 historieundervisningen som præget af "en gammel tradition om at repetere nationalt vedtagne fortællinger om fædrelandet" og påpegede, at "børnene får ikke lov at mærke historien; den skal bare læres" (Rønhede, 2017). Selvom der er tale om et debatindlæg, stemmer kritikken overens med international forskning. Elever engageres især, når historieundervisningen gøres relevant for deres egen livsverden, og når den inddrager kontekstualisering, argumentation og samarbejde (Van Doorselaere, 2025). Samtidig viser en voksende mængde studier, at spilbaserede oplevelser kan øge både motivation og historieforståelse markant. Et studie af det AR-understøttede spil Exploring Ancient Greece fandt, at elever i alderen 10-14 år forbedrede deres historieviden signifikant og vurderede spillet som en foretrukken læringsform (Bekas og Xinogalos, 2024). En metaanalyse dækkende 39 studier konkluderede desuden, at digitale læringsspil generelt er mere motiverende end traditionel undervisning, selvom effekten på faktisk videnstilegnelse er blandet (Wouters m.fl., 2013). Projektet samarbejder med Viborg Katedralskole. Skolen skal i forbindelse med bygningens 100-års jubilæum fejre noget. Gennem samtaler med undervisere har projektgruppen erfaret, at der på skolen er begejstring for et digitalt spil, der inddrager eleverne aktivt og skaber en fælles oplevelse omkring skolens historie.

En central del af problemfeltet er at forstå, hvornår et spil er den rigtige ramme for historieformidling. Mayer, Makransky og Terkildsen har påpeget, at der stadig mangler empirisk evidens for, at øget teknologisk immersion i sig selv skaber bedre læring (Mayer m.fl., 2022). I deres metaundersøgelse rapporterede studerende højere tilstedeværelse i immersive VR-betingelser, men lærte mindre end i mindre immersive formater. Her adskiller projektets ambition sig: målet er ikke primært faktisk indlæring. En nyere analyse af 118 publikationer underbygger, at spilbaseret læring har et markant potentiale for at styrke elevers engagement og historiske forståelse (Lai og Hu, 2025).

Projektets problemformulering er derfor:

Hvordan kan Viborg Katedralskole bruges som fundament for en digital oplevelse i forbindelse med bygningens 100-års jubilæum?

- Hvordan kan bygningen fremvises i et historisk perspektiv?
- Hvordan kan nuværende katedralskole elever inddrages og samarbejde i forbindelse med detalje- og kreative tilføjelser?
- Hvordan kan en digital oplevelse designes med udgangspunkt i en specifik historisk begivenhed fra skolens historie?

3 Hovedafsnit

3.1 Metoder

I projektet blev der anvendt en kombination af agile udviklingsmetoder, tekniske udviklingsværktøjer samt eksperimenterende og iterative arbejdsmetoder for at udvikle et multiplayer-spil baseret på Viborg Katedralskole's historie og arkitektur. Projektet kombinerede softwareudvikling, 3D-modellering, netværksprogrammering og brugeroplevelsesdesign, hvilket gjorde det nødvendigt at arbejde tværfagligt og iterativt gennem hele udviklingsforløbet.

3.1.1 Scrum-lignende udviklingsmetode

Gruppen arbejdede gennem hele projektet ud fra en Scrum-lignende metode med fokus på agil udvikling og iterative sprint. Denne metode blev valgt, da projektet omfattede mange eksperimenterende og teknisk komplekse elementer, hvor krav og løsninger løbende ændrede sig under udviklingen. Ved at arbejde i sprint kunne gruppen kontinuerligt evaluere fremdrift, prioritere opgaver og tilpasse projektets retning efter nye erfaringer og tekniske udfordringer.

Der blev anvendt Planning Poker til estimering af use cases og story points, hvilket gjorde det muligt at vurdere arbejdsbyrde og kompleksitet i de enkelte opgaver. Projektets backlog blev løbende opdateret, og opgaver blev fordelt mellem gruppemedlemmerne ved sprint planning-møder. Efter hvert sprint blev der afholdt retrospektive møder og reviews, hvor gruppen evaluerede både det tekniske arbejde og samarbejdsprocessen. Dette bidrog til en mere struktureret arbejdsproces og gjorde det muligt løbende at forbedre gruppens arbejdsmetoder.

Gruppen valgte dog bevidst ikke at følge fuld Scrum-metodologi. Blandt andet blev daglige stand-up møder fravalgt, da gruppen ønskede en mere fleksibel arbejdsstruktur. Dette gav større frihed i arbejdet, men medførte samtidig udfordringer i forhold til løbende koordinering og opfølgning mellem gruppemedlemmerne.

3.1.2 Teknologier og udviklingsværktøjer

Spillet blev udviklet i Unreal Engine, som blev valgt på grund af engine'ens stærke understøttelse af multiplayer-funktionalitet, avanceret grafik og integration mellem C++ og Blueprint-systemet. Unreal Engine gjorde det muligt at kombinere visuel scripting gennem Blueprints med mere avanceret programmering i C++, hvilket gav gruppen fleksibilitet i udviklingsprocessen.

Projektet blev oprettet som et C++-baseret First Person-projekt, da gruppen ønskede at udvikle spillet fra et førstepersonsperspektiv for at skabe en mere immersiv oplevelse. Der blev tidligt eksperimenteret med samspillet mellem C++ og Blueprints for at finde en balance mellem hurtig prototyping og mere struktureret kodeudvikling.

Til versionsstyring og samarbejde blev der anvendt GitHub. Dette gjorde det muligt for gruppemedlemmerne at arbejde parallelt på projektet, samtidig med at ændringer kunne spores og håndteres gennem pull requests og code reviews. Denne metode blev anvendt for at reducere risikoen for konflikter i projektfilerne og sikre en mere stabil udviklingsproces.

3.1.3 Photogrammetry og map-udvikling

For at skabe et realistisk og autentisk map blev der anvendt photogrammetry som metode til 3D-modellering. Gruppen optog 360-graders video af skolens relevante områder ved hjælp af specialiseret kameraudstyr. Videoerne blev efterfølgende opdelt i stillbilleder, som blev anvendt som datagrundlag i RealityScan udviklet af Epic Games.

RealityScan analyserede billedernes position og vinkler og genererede herefter point clouds, som blev brugt til at skabe 3D meshes og materialer. Denne metode gjorde det muligt at opnå et højt detaljeringsniveau og skabe realistiske gengivelser af skolens omgivelser.

Metoden medførte dog også udfordringer. Flere meshes krævede manuel efterbehandling, collision-opsætning og sammensætning i Unreal Engine. Derudover viste photogrammetry sig at være mindre egnet til meget store sammenhængende områder, hvilket resulterede i fejl og uregelmæssigheder i nogle modeller.

3.1.4 Dataindsamling og samarbejde med fagpersoner

Som en del af projektets researchfase blev der afholdt møder med historielærere fra Viborg Katedralskole for at indsamle viden om skolens historie og rolle under Anden Verdenskrig. Disse samtaler blev anvendt som inspirations- og datagrundlag for spillets narrativ og gameplay-elementer.

Gruppen undersøgte desuden eksisterende multiplayer-spil og gameplay-strukturer for at analysere, hvordan asymmetriske holdbaserede spil fungerer i praksis. Denne analyse blev brugt til at inspirere designvalg omkring teamwork, roller og win conditions i spillet.

3.1.5 Etiske overvejelser

Projektet bygger på historiske begivenheder fra Anden Verdenskrig og inkluderer derfor temaer omkring modstandsbevægelsen og nazistiske styrker. Gruppen har i udviklingen forsøgt at behandle dette emne respektfuldt og med fokus på historisk inspiration frem for glorificering. Formålet har været at skabe en engagerende og lærerig oplevelse med udgangspunkt i skolens historie.

Ved optagelse af billeder og 360-graders video på skolens område blev der taget hensyn til omgivelserne og indhentet nødvendige tilladelser til brug af materialet i projektet.

3.2 Analyse

oprstarten projektet startede med møder med et udvalg fra viborg katadral skole, som udgjorde kunden for projketet i sammenarbejde med dem, blev der nedsat en række tanker og ønsker og krav til hvad skulle være inkluderet i spillet, efter at havde fået krav fra kunden, gik gruppen igang med at analysere kravenden ud udarbejde en række UserStories til spillet

3.2.1 UserStories

ens logo består af tre nøgler, som krydser hinanden. Dette symbol stammer fra en historisk fortælling om en gammel skattekiste, som krævede tre nøgler for at blive

åbnet. Disse nøgler repræsenterede henholdsvis rektoren, en repræsentant fra kirken og en repræsentant fra staten. Denne historiske fortælling blev vurderet som et stærkt gameplay-element og blev derfor også integreret i projektets user stories som centrale mekanikker.

Derudover introducerede kunden viden om skolens rolle under Anden Verdenskrig. Dette førte til use cases, hvor spillerne kan vælge at spille som enten modstandsfolk eller nazister, hvilket danner grundlag for spillets asymmetriske multiplayer-design og forskellige roller i gameplayet.

Som en del af semesterets overordnede krav blev det desuden et fokusområde at implementere online multiplayer-funktionalitet. Dette resulterede i use cases relateret til oprettelse og tilslutning af spil-sessioner samt lobby-systemer. Derudover blev der inkluderet krav om brugerhåndtering, herunder oprettelse af brugere og login-funktionalitet.

Alle disse krav og idéer blev samlet i en samlet liste over user stories:

1. Opret bruger Som spiller vil jeg kunne oprette en konto, så mine fremskridt og statistikker kan gemmes.
2. Login Som spiller vil jeg kunne logge ind, så jeg kan spille online med andre.
3. Join lobby Som spiller vil jeg kunne join en lobby, så jeg kan deltage i et spil med andre spillere.
4. Holdvalg Som spiller vil jeg kunne vælge hold, så jeg enten kan spille som modstandsmand eller SS-soldat.
5. Bevæge sig skjult Som modstandsmand vil jeg kunne snige mig rundt på kortet uden at blive opdaget.
6. Finde nøgler Som modstandsmand vil jeg kunne finde skjulte nøgler rundt omkring på skolen.
7. Åbne kiste Som modstandsmand vil jeg kunne bruge de 3 nøgler til at åbne kisten og vinde spillet.

8. Samarbejde Som modstandsmand vil jeg kunne kommunikere med mit hold for at koordinere strategier.
9. Patruljere området Som SS-soldat vil jeg kunne patruljere skolens områder for at finde modstandsfolkene.
10. Fange spillere Som SS-soldat vil jeg kunne stoppe eller fange modstandsfolkene.
11. Overvåge områder Som SS-soldat vil jeg kunne overvåge bestemte områder for at beskytte nøglerne eller kisten.
12. Vælge klasse Som spiller vil jeg kunne vælge mellem forskellige klasser før spilstart.
13. Unikke evner Som spiller vil jeg kunne bruge specielle evner, som er unikke for min klasse.
14. Timer Som spiller vil jeg kunne se, hvor meget tid der er tilbage af kampen.
15. Victory conditions Som spiller vil jeg tydeligt kunne se, hvornår et hold har vundet.
16. Befri fangede holdkammerater Som modstandsmand vil jeg kunne befri fangede holdkammerater, så de kan være med igen.
17. Voice chat Som spiller vil jeg kunne kommunikere via voice chat med de andre spillere.
18. Map Som spiller vil jeg have et map, der reflekterer det historiske perspektiv.
19. NPC Som spiller vil jeg kunne spille mod NPC'er, hvis der ikke er nok spillere.
20. Opret session Som spiller vil jeg kunne oprette en session, så andre spillere kan joine mit spil.

Efter udarbejdelsen af user stories anvendte gruppen Planning Poker som estimeringsmetode. Her blev hver use case tildelt story points baseret på vurderet kompleksitet og arbejdsbyrde. Dette gjorde det muligt at planlægge sprintforløb mere struktureret og skabe et bedre overblik over projektets samlede omfang.

Undervejs i projektet blev story point-estimerne løbende revideret og justeret i takt med, at gruppen fik mere indsigt i de tekniske udfordringer og opgavernes reelle kompleksitet. Den endelige fordeling af story points fremgår af figuren nedenfor:

Problem	User Story	Story Points
1	Som spiller vil jeg kunne oprette en konto, så mine fremskridt og statistikker kan gemmes	40
2	Som spiller vil jeg kunne logge ind, så jeg kan spille online med andre.	5
3	Som spiller vil jeg kunne join en lobby, så jeg kan deltage i et spil med andre spillere	5
4	Som spiller vil jeg kunne vælge hold, så jeg enten kan spille som modstandsmand eller SS-soldat.	8
5	Som modstandsmand vil jeg kunne snige mig rundt på kortet uden at blive opdaget.	3
6	Som modstandsmand vil jeg kunne finde skjulte nøgler rundt omkring på skolen.	8
7	Som modstandsmand vil jeg kunne bruge de 3 nøgler til at åbne kisten og vinde spillet.	3
8	Som modstandsmand vil jeg kunne kommunikere med mit hold for at koordinere strategier.	13
9	Som SS-soldat vil jeg kunne patruljere skolens områder for at finde modstandsfolkene	2
10	Som SS-soldat vil jeg kunne stoppe eller fange modstandsfolkene	13
11	Som SS-soldat vil jeg kunne overvåge bestemte områder for at beskytte nøglerne eller kisten.	2
12	Som spiller vil jeg kunne vælge mellem forskellige klasser før spilstart.	8
13	Som spiller vil jeg kunne bruge specielle evner, som er unikke for min klasse.	20

Problem	User Story	Story Points
14	Som spiller vil jeg kunne se, hvor meget tid der er tilbage af kampen.	5
15	Som spiller vil jeg tydeligt kunne se, hvornår et hold har vundet.	3
16	Som modstandsmand vil jeg gerne kunne befri eventuelt fangede holdkammerater, så de kan være med igen	5
17	Som spiller vil jeg gerne kunne kommunikere via voice chat med de andre	40
18	Som spiller vil jeg gerne have et map der reflekterer det historiske perspektiv	8
19	Som spiller vil jeg gerne kunne spille mod NPC hvis der ikke er nok spillere	100
20	Som spiller vil jeg kunne oprette session, så andre spillere kan joine mit spil	5

Tabel 1: User stories og story points

3.2.2 Ikke funktionelle krav

Ud over user stories blev der også defineret en række ikke-funktionelle krav, som har haft til formål at beskrive de overordnede kvalitetsmæssige og tekniske rammer for systemet. Hvor user stories fokuserer på funktionalitet og brugerens interaktion med spillet, beskriver de ikke-funktionelle krav de egenskaber, som systemet skal leve op til i forhold til ydeevne, stabilitet, brugervenlighed og skalerbarhed.

Disse krav tager udgangspunkt i, at spillet skal kunne fungere stabilt i en online multiplayer-kontekst, hvor flere spillere interagerer simultant i realtid. Dette stiller særlige krav til netværksperformance, serverhåndtering og synkronisering mellem klienter. Samtidig er spillet udviklet med henblik på skolens elever som primær målgruppe, hvilket betyder, at brugeroplevelsen skal være intuitiv, let tilgængelig og hurtigt forståelig, også for brugere uden tidligere erfaring med lignende spil.

De ikke-funktionelle krav fungerer derfor som en række retningslinjer, der sikrer, at

spillet ikke blot fungerer teknisk, men også lever op til forventningerne om spilbarhed, læringskurve og stabil drift i et undervisnings- og testmiljø. Dette inkluderer blandt andet krav til responstid, netværksforsinkelse, systemstabilitet og brugergrænsefladens kompleksitet, som alle er centrale faktorer i udviklingen af et multiplayer-spil baseret på Unreal Engine.

Derudover har kravene også fungeret som et styringsværktøj i udviklingsprocessen, da de har givet gruppen et klart grundlag for at vurdere tekniske løsninger og prioritere implementeringer ud fra, hvad der var realistisk inden for projektets tidsramme og tekniske begrænsninger.

- Spillet skal reagere på spillerinput inden for 100 ms under normale netværksforhold.
- Serveren skal kunne håndtere mindst 6 samtidige spillere uden nedbrud.
- Brugerfladen skal være let at forstå for nye spillere.
- Nye spillere skal kunne lære de grundlæggende mekanikker på under 5 minutter.
- Systemet skal kunne opdage eller forhindre simple former for cheating.
- Alle spilleres positioner skal opdateres i realtid med minimal forsinkelse.
- Spillet skal understøtte asynkrone handlinger mellem spillere og server.
- Kodebasen skal være opdelt i moduler, så nye klasser og maps nemt kan tilføjes.
- Spillet skal være lavet i Unreal Engine.
- Klasserne i spillet skal være velbalancerede.
- Voice chatten skal fungere effektivt og kun mellem hold, samt der skal kunne lægges filtre over voice chatten.

3.2.3 Usecase Diagram

Det overordnede use case diagram illustrerer, hvordan spillets funktionalitet er opbygget som en sammenhængende brugerrejse, hvor de enkelte use cases enten er afhængige af hinanden eller udvider hinandens funktionalitet. Systemet er designet som en sekventiel proces for spilleren, hvor der først etableres adgang til spillet, hvorefter selve gameplay-loopet aktiveres.

Processen starter med use casene *Opret bruger* og *Login*, som tilsammen udgør den grundlæggende adgang til systemet. Oprettelse af bruger gør det muligt at gemme spillerdata, mens login sikrer, at spilleren kan identificeres og tilgå sin profil. Disse to use cases fungerer som forudsætning for alle efterfølgende funktioner, da ingen gameplay-relaterede handlinger kan udføres uden autentificering.

Herefter følger use casen *Join lobby*, hvor spilleren tilslutter sig en aktiv session. Denne use case fungerer som bindeled mellem login-systemet og selve spiloplevelsen, da den placerer spilleren i et multiplayer-miljø sammen med andre deltagere. Når spilleren er i en lobby, aktiveres næste centrale use case: *Holdvalg*. Her fordeles spilleren på enten modstandsbevægelsen eller SS-soldaterne, hvilket definerer spillerens rolle og efterfølgende gameplay-mekanikker.

Efter holdvalg udvides systemet yderligere med use casen *Vælg klasse*, som giver spilleren mulighed for at specialisere sig inden for sit hold. Dette use case leder videre til *Unikke evner*, som er direkte afhængige af den valgte klasse og dermed fungerer som en udvidelse af klassevalget (include-relation).

Når spillet er startet, aktiveres selve gameplay-loopet, som er forskelligt afhængigt af hold:

For modstandsmanden inkluderer dette:

- Bevæge sig skjult, som er en grundlæggende bevægelsesmekanik, der understøtter stealth gameplay.
- Finde nøgler, som er en central progressionsmekanik.
- Åbne kiste, som er den direkte win condition for holdet og afhænger af, at nøglerne er indsamlet.
- Samarbejde, som understøtter koordinering mellem spillere.

- Fangede holdkammerater, som giver mulighed for at befri allierede og genindføre dem i spillet.

For SS-soldaten består gameplayet af:

- Patruljere området, som danner basis for bevægelse og kontrol af kortet.
- Overvåge områder, som giver mulighed for strategisk kontrol over nøgleområder.
- Fange spillere, som fungerer som en direkte modmekanik til modstandsbevægelsen.

Disse mekanikker er designet som parallelle gameplay-strukturer, hvor begge hold har forskellige mål, men interagerer i det samme system.

Derudover er der en række systemoverordnede use cases, som understøtter hele spiloplevelsen:

- Timer, som løbende informerer spilleren om spillets resterende tid.
- Victory conditions, som aktiveres når enten timer eller spilmål er opfyldt og tydeligt informerer spilleren om kampens resultat.
- Voice chat, som understøtter kommunikation mellem spillere og er begrænset til holdvis interaktion.
- Mappet, som repræsenterer spillets fysiske miljø og er designet ud fra skolens historiske bygninger.
- NPC, som aktiveres i situationer hvor der ikke er nok spillere til at understøtte multiplayer gameplay.

Endelig findes systemfunktionelle use cases som:

- Opret session, som gør det muligt for en spiller at hoste et spil, som andre kan deltage i.

Sammenhæng i systemet:

Set samlet fungerer use casene som en lagdelt struktur, hvor systemet starter med brugeradgang (login), fortsætter med session og lobby-system, derefter rollefordeling (hold og klasse), og til sidst det egentlige gameplay-loop med mål, progression og win condition.

Diagrammet viser dermed en klar afhængighedskæde, hvor de tidlige use cases er nødvendige for at aktivere de senere gameplay-funktioner, og hvor de centrale spil-mekanikker er opdelt i holdspecifikke handlinger, der tilsammen skaber den samlede spiloplevelse.

Diagrammets enkelte use cases er udviklet ved at sammensmelte flere user stories ned til enkelte use cases for systemet. Dette er samlet endt ud i dette use case-diagram.

3.2.4 Actor Discription

3.2.5 Use case beskrivelser

3.2.6 Aktivitets diagrammer

3.2.7 System sequence diagrammer

3.2.8 State mashine diagrammer

3.2.9 Domaine model

3.3 Design

3.3.1 Klasse diagram

Ud fra domænemodellen er der blevet udarbejdet et klassediagram, som repræsenterer en mere teknisk og implementeringsnær struktur af systemets komponenter og deres indbyrdes relationer. Hvor domænemodellen primært havde til formål at beskrive de overordnede begreber, aktører og systemets funktionelle områder på et abstrakt niveau, fungerer klassediagrammet som en konkretisering af disse elementer i form af klasser, attributter, metoder og relationer. Overgangen fra domænemodel til klassediagram har derfor bestået i en systematisk nedbrydning af de centrale begreber fra domænet til mere tekniske enheder, dog skal det pointeres, at grunden brug af unreal

enginge så er selve klasse diagrammet markant anderledes end domaine modellen, da Unreal Engine kommer med en færdig bygget arkitektur som man skal bruge for at bygge sit system op omkring, dog er gameplay-elementer, spillertyper, sessionshåndtering og netværkslogik blevet omsat til konkrete klasser med ansvar og funktioner.

I denne proces som dette har det været nødvendigt at skelne mellem gameplay-logik, brugerinteraktion er der blevet identificeret hvilke dele af domænet der skal repræsenteres som selvstændige klasser, hvilke der skal modelleres som relationer, og hvilke der skal håndteres som arv eller komposition. Særligt i et spiludviklingsprojekt og systemmæssige funktioner såsom netværk og state management. Klassediagrammet afspejler derfor både spillets struktur og de tekniske afhængigheder, som opstår i implementeringen af et multiplayer-system.

Det er dog vigtigt at bemærke, at klassediagrammet i dette tilfælde har en begrænset praktisk overgang fra analyse til design i forhold til den faktiske implementering i Unreal Engine. Unreal Engine bygger på en allerede etableret og fast arkitektur, hvor centrale systemer som Actors, Components, Controllers og GameMode allerede er defineret af engine'en selv. Dette betyder, at man i praksis ikke frit designer hele sin klassearkitektur fra bunden, men i stedet arbejder inden for Unreal Engines eksisterende framework og nedarver fra deres baseklasser. Udviklingsarbejdet består derfor i høj grad i at udvide og specialisere disse klasser fremfor at opbygge et komplet klassesdesign fra bunden.

Som konsekvens heraf fungerer klassediagrammet primært som et dokumentarisk værktøj, der kan bruges til at forstå systemets logik og struktur på et højt niveau, snarere end som en direkte Processværktøj. Det giver et overblik over sammenhængen mellem systemets dele, men afspejler ikke nødvendigvis den fulde Design fase. Klassediagrammet skal derfor ses som en abstraheret model, der er nyttig til at forstå og tale om systemet, men har ikke kunne direkte designes ud fra Analysen, uden at tage højde for Unreal Engines enlige struktur og arkitektur der allerede er defineret af Unreal Engine.

Som det fremgår af klassediagrammet (bilag D.26), er der en række arvrelationer fra de underliggende systemer markeret som "Unreal Engine"-elementer. Disse er inkluderet for at tydeliggøre, hvilke dele af systemets struktur der stammer fra Unreal Engine, og som derfor ikke er fuldt dokumenteret i diagrammet. Formålet med mar-

keringen er samtidig at illustrere engineens arkitektur samt hvilke komponenter, der er leveret af engineen, og hvilke der er udviklet ovenpå denne.

Derudover er de enkelte elementer i diagrammet annoteret efter implementeringstype. Elementer markeret med “Blueprint” angiver dele, der er udviklet i Unreal Engines Blueprint-system, mens klasser markeret med “C++” repræsenterer systemer, som er implementeret af gruppen i C++-kode. Denne opdeling er foretaget for at skabe et klart overblik over, hvilke dele af systemet der er udviklet med hvilke teknologier, og dermed tydeliggøre den tekniske opbygning og ansvarfordeling i projektet.

3.3.2 Sequence diagram

Som det fremgår af sekvensdiagrammet (bilag D.29), fungerer diagrammet som et samlet design for “Sunshine”-systemets sekvenser, der er konsolideret fra en række mindre og mere analytiske systemsekvensdiagrammer. Hvor de individuelle systemsekvensdiagrammer hver især beskriver isolerede brugerinteraktioner og specifikke funktionelle flows, samler dette overordnede sekvensdiagram hele systemets interaktioner i én sammenhængende model.

Formålet med at udarbejde et samlet sekvensdiagram er at skabe et helhedsorienteret overblik over systemets dynamiske adfærd. Dette gør det muligt at se, hvordan de forskellige dele af systemet interagerer på tværs af funktioner, og hvordan data og kontrolflow bevæger sig mellem aktører og systemkomponenter. I praksis fungerer diagrammet som en bro mellem de tidlige analysemodeller og den endelige design- og implementeringsfase.

I processen med at udarbejde diagrammet er de individuelle systemsekvensdiagrammer blevet analyseret og abstraheret, hvorefter overlappende og relaterede interaktioner er blevet samlet i fælles sekvenser. Dette har krævet en strukturering af de forskellige hændelsesforløb, så redundans er fjernet, og så de mest centrale interaktionsmønstre fremstår tydeligt. Resultatet er et mere kompakt og sammenhængende diagram, som stadig bevarer den logiske struktur fra de oprindelige analyser.

Det samlede sekvensdiagram gør det samtidig muligt at identificere afhængigheder mellem systemets komponenter samt rækkefølgen af vigtige operationer. Dette er særligt vigtigt i forhold til forståelsen af systemets arkitektur, da det tydeliggør, hvordan de forskellige moduler i “Sunshine”-systemet kommunikerer og samarbejder

under udførelsen af centrale funktioner.

Derudover bidrager diagrammet til at sikre konsistens mellem design og implementering, da det fungerer som referencepunkt for udviklingen. Ved at samle alle sekvenser i ét overordnet diagram opnås en mere præcis visualisering af systemets samlede adfærd, hvilket både styrker dokumentationen og gør det lettere at validere systemets funktionelle krav.

diagrammet er nærmest opdelt i 2 forskellige sekvenser, den øvre sekvens som udgøre hele login fasen frem til at spilleren har valgt en karakter, og så den nedre fase, som som er markeret som et Loop, dette loop anses som Game loopet, og disse sekvenser kører derfor hele tiden, ind til spillet stopper, denne opdelingen hænger også godt sammen med spillets opdeling i en menu struktur for at være klar til at spille, og så selve spillet

som det også ses fra sequence diagrammet er systemet oprindeligt designet til også at inkludere en Database, som skulle administrere bruger logins, samt bruger data, således at spiller kunne logge ind i systemet, og hente navn og score, databasen er konceptuelt designet men er ikke blevet implementeret grundet prioritering af udvikling af selve spillet først

3.3.3 Conceptual ER-Diagram

Som det fremgår af ER-diagrammet (bilag D.27) er der designet en database, som understøtter et login-system og håndteringen af de centrale data i systemet. Databasen er udviklet med udgangspunkt i et klassisk databasedesign-forløb, hvor der først er blevet udarbejdet et Entity-Relationship (ER) diagram som fundament for den endelige databaseimplementering.

Udarbejdelsen af ER-diagrammet er sket på baggrund af domænemodellen (bilag D.24), hvor de væsentligste entiteter i systemets problemområde først er blevet identificeret. Domænemodellen har fungeret som et konceptuelt udgangspunkt, der beskriver systemets overordnede struktur og forretningsmæssige begreber. Herefter er disse entiteter blevet analyseret og omsat til en mere databaseret model, hvor fokus er på datarepræsentation og relationer frem for abstrakte domænebegreber.

Efter identifikationen af entiteter er der blevet foretaget en detaljeret analyse af de enkelte entiteters attributter. Formålet med denne fase har været at definere, hvilke

egenskaber der er nødvendige for at repræsentere hver entitet korrekt i databasen. Dette inkluderer blandt andet overvejelser om primærnøgler, datatyper samt hvilke attributter der er obligatoriske eller valgfrie. Denne strukturering er essentiel for at sikre dataintegritet og entydighed i databasen.

Dernæst er relationerne mellem entiteterne blevet analyseret og modelleret i ER-diagrammet. Her er der taget stilling til kardinalitet (1:1, 1:N og N:M), blandt andet ses det at en kamp indholder 2 hold, eller så er det en 1:1 relation der er imellem alle entiteterne. Denne del af designprocessen er central, da den sikrer, at databasen kan understøtte systemets logiske sammenhænge uden redundans eller inkonsistens i data. I tilfælde hvor der opstår mange-til-mange-relationer, er disse blevet håndteret ved introduktion af forbindelsestabeller for at normalisere datastrukturen.

Det endelige ER-diagram fungerer dermed som en samlet konceptuel model for databasen, hvor både strukturen af data og relationerne mellem dem er defineret.

dog som før nævnt er denne database ikke implementeret, dog er den videre analyseret til et EER

3.3.4 EER-Diagram

Efter færdiggørelsen af ER-diagrammet er der blevet udarbejdet et EER-diagram (bilag D.28). Dette diagram udgør en videreudvikling af det oprindelige ER-diagram og repræsenterer et Enhanced Entity-Relationship (EER) design, hvor modellen udvides med mere avancerede datamodelleringskoncepter. Formålet med EER-diagrammet er at give en mere detaljeret og implementeringsnær beskrivelse af databasen, hvor både strukturelle relationer og yderligere constraints er eksplicit defineret.

I EER-diagrammet indgår blandt andet mapping af entiteterne i form af primærnøgler og fremmednøgler. Hver entitet er således blevet tildelt en primærnøgle markeret med PK, som entydigt identificerer hver instans af entiteten i databasen. Et eksempel herpå er entiteten *spiller*, hvor attributten *spiller_id* anvendes som primærnøgle. Denne nøgle er et såkaldt "opfundet id" (surrogate key), som genereres og kontrolleres af systemet selv. Fordelen ved denne type nøgle er, at den er stabil, unik og ikke afhænger af forretningsdata, hvilket betyder, at den ikke ændres over tid og derfor er velegnet som identifikator i relationelle databaser.

Derudover er fremmednøgler markeret med FK, hvilket angiver relationer mellem

entiteterne. Fremmednøgler anvendes til at sikre referentiel integritet og etablere forbindelser mellem tabeller. Dette ses eksempelvis i entiteten *spillerkamp*, som fungerer som en associerende entitet. Denne type entitet anvendes til at håndtere relationer mellem to eller flere entiteter og indeholder derfor fremmednøgler fra flere forskellige tabeller. På denne måde fungerer *spillerkamp* som en kobling, der binder data fra relaterede entiteter sammen og muliggør modellering af komplekse relationer såsom mange-til-mange-sammenhænge.

EER-diagrammet indeholder desuden udvidelser i form af specialisering og generalisering (nedarvning), hvilket ikke eksplicit fremgår af det oprindelige ER-diagram. Dette ses eksempelvis ved entiteten *klasse*, hvor der er defineret en hierarkisk struktur mellem overordnede og underordnede entitetstyper. Diagrammet angiver her både mandatory participation og OR-kardinalitet, hvilket betyder, at en given instans af den overordnede entitet nødvendigvis skal være specialiseret som præcis én af de underliggende entitetstyper. Denne constraint sikrer en streng klassifikation af data og forhindrer, at entiteter eksisterer uden en defineret subtype.

Tilsvarende gør sig gældende for *hold*-entiteten, hvor der ligeledes er anvendt specialisering. Dette bidrager til en mere præcis datamodel, hvor forskellige typer af hold kan repræsenteres uden at kompromittere datakonsistens eller struktur. Ved at anvende EER-modellens udvidede begreber opnås dermed en mere nuanceret og realistisk datamodellering, som bedre afspejler systemets domæne og de regler, der gælder heri.

Samlet set fungerer EER-diagrammet som en udvidet og mere formaliseret version af ER-diagrammet, hvor både nøgler, relationer og specialiseringer er tydeligt defineret.

3.3.5 Package Diagram

Package Diagrammet (bilag D.25) er et supplerende diagram, som gruppen har valgt at anvende for at skabe et overblik over systemets overordnede arkitektur. Behovet for dette diagram opstod som følge af, at systemet er opbygget omkring flere forskellige delsystemer, herunder en database, selve spillet udviklet i Unreal Engine samt eksterne værktøjer såsom RealityScan.

Diagrammet illustrerer den lagdelte struktur i systemet og viser, hvordan de for-

skellige komponenter er organiseret og afhængige af hinanden. Dette har været med til at tydeliggøre, hvordan data og funktionalitet flyder mellem de enkelte dele af systemet under udviklingen.

Formålet med Package Diagrammet har derfor været at skabe et arkitektonisk overblik, som understøtter forståelsen af systemets opbygning og gør det lettere at udvikle og vedligeholde de enkelte moduler i projektet.

3.4 Implementation

Den detaljerede implementering af systemets forskellige dele samt forklaringer af koden er uddybet i procesrapporten, da dette vurderes som en del af udviklingsprocessen. I dette afsnit fokuseres der derfor på, hvordan forskellige implementeringsmetoder og koncepter er blevet anvendt, samt hvilken betydning disse valg har haft for det endelige produkt.

3.4.1 Brug af OOP

Projektet har i høj grad gjort brug af koncepter fra objektorienteret programmering (OOP), særligt principperne omkring nedarvning. Som tidligere nævnt arbejdes der i Unreal Engine ud fra en struktur, hvor engine'en allerede stiller en række baseklasser til rådighed. Disse baseklasser danner fundamentet for udviklingen, hvorfra der nedarves og videreudvikles funktionalitet til projektets egne behov.

Ved brug af nedarvning har det været muligt at genbruge eksisterende funktionalitet fra Unreal Engines baseklasser, samtidig med at systemet kunne tilpasses projektets specifikke krav. Dette har blandt andet betydet, at fælles funktionalitet kunne implementeres centralt i overordnede klasser og derefter genbruges i underklasser. På den måde blev udviklingen mere struktureret og vedligeholdelsesvenlig, da ændringer i en baseklasse automatisk kunne videreføres til de tilhørende underklasser.

Derudover har anvendelsen af objektorienterede principper bidraget til en mere modular opbygning af systemet. Dette gjorde det lettere at udvide projektet med nye funktioner og objekter uden at skulle omskrive eksisterende kode. Særligt i udviklingen af spillets forskellige aktører og systemer har OOP-principperne haft stor betydning for at skabe et fleksibelt og skalerbart system.

3.4.2 Netværk

Projektet har i høj grad gjort brug af Unreal Engines netværksarkitektur og replication-system for at muliggøre multiplayerfunktionalitet. Unreal Engine er bygget op omkring en klient-server-model, hvor serveren fungerer som den autoritative instans, der håndterer spillets logik og synkroniserer data mellem de tilsluttede klienter. Dette har været centralt i udviklingen af projektets multiplayerfunktioner.

Ved brug af replication har det været muligt automatisk at synkronisere objekter, variabler og handlinger mellem serveren og de forskellige klienter i spillet. Dette har blandt andet været anvendt til spillerpositioner, animationer og andre gameplay-relaterede elementer, så alle spillere oplever den samme spilstilstand. Unreal Engines indbyggede replication-system har dermed reduceret behovet for manuel håndtering af netværkskommunikation, og derved også abstraheret TCP og UDP transportprotokoller væk fra udviklingen og gjort implementeringen mere overskuelig.

Derudover har projektet gjort brug af Remote Procedure Calls (RPC'er), som gør det muligt at udføre funktioner på enten serveren eller specifikke klienter. Dette har været nødvendigt i situationer, hvor spillerhandlingerne skulle valideres af serveren, før de blev videreformidlet til andre spillere. På den måde sikres det, at serveren bevarer kontrollen over spillets tilstand, hvilket både forbedrer stabiliteten og reducerer risikoen for fejl og usynkroniserede handlinger mellem klienterne.

Anvendelsen af netværks- og replication-koncepter har derfor haft stor betydning for udviklingen af et stabilt multiplayer-system, hvor flere spillere kan interagere i samme spilverden med en ensartet oplevelse på tværs af klienterne.

3.4.3 C++

C++ er anvendt på to forskellige måder i Unreal Engine, da den primære implementering af gameplay typisk sker gennem Blueprints. På trods af dette spiller C++ stadig en central rolle i projektets udvikling og bruges særligt, hvor Blueprints ikke er tilstrækkeligt fleksible eller effektive.

Den første anvendelse af C++ er til at opbygge mere avancerede baseklasser end dem, Unreal Engine som standard stiller til rådighed. Dette indebærer blandt andet definition af specifikke variabler og funktionalitet, som er skræddersyet til projektets

behov. Denne tilgang har gjort det muligt at skabe mere specialiserede og kontrollerede systemer, som bedre passer til spillets design og struktur.

Den anden anvendelse af C++ er implementeringen af mere komplekse og performancekritiske metoder, som enten er svære at realisere i Blueprints eller som kræver bedre optimering. Disse funktioner kan derefter stadig tilgås og anvendes gennem Blueprints, hvilket giver en fleksibel arbejdsdeling mellem visuel scripting og programmering. Ved at definere funktioner med kategorier i UFUNCTION kan udviklede metoder desuden organiseres overskueligt i Blueprints, hvilket forbedrer overblikket under udviklingen.

Det er desuden vigtigt at bemærke, at C++ i Unreal Engine adskiller sig fra "rå" C++ programmering. Unreal benytter et reflektionssystem baseret på makroer som UFUNCTION og UPROPERTY, som gør det muligt for engine'en at håndtere funktioner, variabler og deres eksponering til Blueprints. Samtidig betyder dette system, at udvikleren i høj grad ikke selv skal håndtere memory management i form af garbage collection, da Unreal Engine automatisk håndterer oprydning af objekter.

Brugen af C++ med pointers giver samtidig en effektiv måde at referere til objekter på, da det undgår unødvendige kopier i hukommelsen. Dette bidrager til en mere optimeret og performancedrevet struktur sammenlignet med nogle højere niveausprog, hvor objekthåndtering ofte sker mere indirekte.

3.4.4 Blueprints

3.5 Tests

3.5.1 Test Cases

På baggrund af use casene blev der løbende udarbejdet test cases til systemet. Disse test cases blev skrevet før implementeringen af koden med inspiration fra principperne bag Test-Driven Development (TDD). Formålet var at sikre, at systemet løbende kunne vurderes op imod de opstillede krav og forventede funktionaliteter.

Testene (bilag H) er opdelt i forskellige kategorier og fokuserer på specifikke dele af systemet. Under implementeringsfasen blev testene løbende anvendt og udført af systemets udviklere for at verificere funktionaliteten.

Efterfølgende blev samtlige tests udført af et testpanel bestående af to personer uden for projektgruppen. Det skal dog bemærkes, at begge testpersoner har relationer til medlemmer af gruppen, hvorfor der bør tages forbehold for potentiel bias i evalueringen. Testpanelet blev derfor instrueret i at forholde sig neutralt og objektivt til deres oplevelse af spillet samt til de krav, som testene opstillede.

resultatet af testpanelets gennemgang af testene, kan ses under (bilag H), ud fra den gældende trin i testen, er en status beskrevet, for om testen er godkendt eller fejlet

3.6 Resultater

På baggrund af hele processen kan det konkluderes, at gruppen har været i stand til at udvikle et spil i Unreal Engine, som indeholder tydelige referencer til Viborg Katedralskole. Projektet har gennem både design-, implementerings- og testfasen udviklet sig til et sammenhængende produkt, hvor de valgte teknologier og metoder er blevet anvendt til at understøtte den ønskede funktionalitet og det overordnede koncept.

De indarbejdede referencer til Viborg Katedralskole kommer til udtryk gennem spillets miljø, strukturer og visuelle elementer, hvilket har været med til at skabe en genkendelig kobling til den virkelige verden. Samtidig har brugen af Unreal Engine, Blueprints og C++ gjort det muligt at realisere projektets idéer i en interaktiv og funktionel spiloplevelse.

Samlet set har projektets iterative udviklingsproces, kombineret med anvendelsen af relevante softwareteknologier og systematiske arbejdsmetoder, gjort det muligt at opnå en prototype af et produkt, som let ville kunne bygges videre og og færdiges, der både teknisk og konceptuelt lever op til projektets oprindelige mål.

4 Diskussion

skriv noget omkring at klasse diagram ikke giver mening når man arbejder i unreal

5 Konklusion og anbefalinger

Litteratur

- Bekas, A., & Xinogalos, S. (2024). Exploring Historical Monuments and Learning History through an Augmented Reality Enhanced Serious Game. *Applied Sciences*, 14, 6556. <https://doi.org/10.3390/app14156556>
- Lai, C.-H., & Hu, P.-Y. (2025). The Gaming Revolution in History Education: The Practice and Challenges of Integrating Game-Based Learning into Formal Education. *Information*, 16, 490. <https://doi.org/10.3390/info16060490>
- Mayer, R., Makransky, G., & Parong, J. (2022). The Promise and Pitfalls of Learning in Immersive Virtual Reality. *International Journal of Human-Computer Interaction*, 39, 1–10. <https://doi.org/10.1080/10447318.2022.2108563>
- Rønhede, S. (2017). Historiefaget er kedsommelig dyrkelse af fædrelandsmyter [Debatindlæg, tilgået via information.dk]. *Information*.
- Van Doorsselaere, J. (2025). Meaningful learning beyond the textbook: a case study of student experiences during an authentic historical inquiry on local heritage. *History Education Research Journal*, 22. <https://doi.org/10.14324/HERJ.22.1.09>
- Viborg Katedralskole. (2026 marts). *Levende historie - Viborg Katedralskole*. Hentet 13. maj 2026, fra <https://viborgkatedralskole.dk/om-vk/levende-historie>
- Wouters, P., Nimwegen, C., Oostendorp, H., & Spek, E. (2013). A Meta-Analysis of the Cognitive and Motivational Effects of Serious Games. *Journal of Educational Psychology*, 105, 249–265. <https://doi.org/10.1037/a0031311>